

Favo

Manual técnico do alocador de memória em C:
arquitetura interna, instalação, API, linha de comando,
painel web e extensão.

Manual técnico · versão 1.0.0

Copyright (©) 2026 Erik Pimenta · licença MIT

Sumário

1. Visão geral
2. Como a memória é organizada
3. Algoritmos de alocação
4. Instalação
5. Biblioteca em C
6. Substituindo o malloc com LD_PRELOAD
7. Linha de comando
8. Painel web
9. API REST e WebSocket
10. Configuração
11. Como estender o Favo
12. Testes
13. Solução de problemas

1. Visão geral

O Favo é um gerenciador de memória heap escrito em C. Em vez de usar o `malloc` e o `free` da biblioteca padrão, ele pede regiões grandes ao sistema operacional e administra sozinho a divisão dessas regiões em blocos entregues ao programa.

O projeto tem quatro partes, todas usando o mesmo núcleo:

Parte	Onde fica	Para que serve
Biblioteca `libfavo`	`include/`, `src/`	Núcleo do alocador: arenas, blocos, lista circular, árvore rubro-negra, verificador
`libfavo_preload.so`	`src/preload.c`	Substitui o malloc de qualquer programa dinâmico no Linux
CLI `favo`	`tools/`	Executa traces, compara algoritmos, mede desempenho, gera traces
API e painel	`api/`, `web/`	FastAPI que chama a biblioteca via ctypes, SQLite para o histórico e painel em TypeScript

Por que é difícil: um alocador trabalha com aritmética de ponteiros sobre memória crua. Um único erro de tamanho corrompe cabeçalhos vizinhos e o programa inteiro cai mais tarde, longe da causa. Por isso o Favo tem um verificador que confere cada invariante e uma suíte de testes com AddressSanitizer, Valgrind e 150 mil operações aleatórias por algoritmo.

2. Como a memória é organizada

2.1 Arenas

Uma arena é uma região contínua pedida ao sistema. O Favo usa três origens:

Origem	Função	Quando
`sbrk`	Move o *program break*, o fim do segmento de dados do processo	Heap global no Linux (padrão do LD_PRELOAD)
`mmap`	Mapeia páginas anônimas em qualquer lugar do espaço de endereços	Heaps criados com `favo_heap_create`, blocos grandes
`VirtualAlloc`	Reserva e confirma páginas no Windows	Qualquer heap no Windows

Cada arena começa com um cabeçalho de 48 bytes (próxima arena, base, tamanho, primeiro bloco, fim e origem), seguido de um **prólogo** de 8 bytes marcado como ocupado. No fim fica um **epílogo** de 16 bytes com tamanho zero, também marcado como ocupado. Prólogo e epílogo funcionam como sentinelas: a fusão de blocos nunca precisa testar se está na borda da arena.

Com `sbrk`, se o break não foi movido por mais ninguém desde o último pedido, a arena é estendida no lugar: o epílogo antigo vira o cabeçalho de um bloco livre novo, que se funde ao último bloco livre da arena.

```
arena
+-------------+---------+-------------------------+---------+-----+----------+
| cabecalho   | prologo | bloco 1                 | bloco 2 | ... | epilogo  |
| da arena    |   8 B   |                         |         |     |   16 B   |
+-------------+---------+-------------------------+---------+-----+----------+
base          base+56   base+64 (payload em base+80, alinhado a 16)
```

2.2 Blocos e boundary tags

Todo bloco tem tamanho múltiplo de 16 e no mínimo 64 bytes.

Campo	Tamanho	Conteúdo
Cabeçalho: `info`	8 B	Tamanho do bloco com os 4 bits baixos usados como flags: bit 0 = ocupado, bit 1 = bloco grande
Cabeçalho: `magic`	4 B	Assinatura: `0xFA70B10C` ocupado, `0xFA70F2EE` livre, `0xFA70ED6E` epílogo
Cabeçalho: `request`	4 B	Quantos bytes o programa pediu (para medir aproveitamento)
Payload	variável	Memória do programa. Começa 16 bytes após o cabeçalho e é sempre alinhado a 16
Rodapé	8 B	Cópia de `info`. Permite achar o bloco anterior em O(1)

O cálculo do tamanho do bloco para um pedido de *n* bytes é `max(64, alinhar16(n + 24))`. A parte útil devolvida por `malloc_usable_size` é o tamanho do bloco menos 24.

2.3 Blocos livres

Quando livre, o espaço do payload guarda os ponteiros das duas estruturas de busca ao mesmo tempo, sem memória extra:

Campo	Estrutura
`next`, `prev`	Lista circular duplamente encadeada
`left`, `right`	Filhos na árvore rubro-negra
`parent_color`	Ponteiro do pai com a cor guardada no bit mais baixo (1 = vermelho)

Esses 40 bytes, somados aos 16 do cabeçalho e aos 8 do rodapé, explicam o bloco mínimo de 64 bytes.

2.4 Divisão (split)

Se o bloco encontrado é maior que o necessário e a sobra tem pelo menos 64 bytes, ele é dividido. A sobra assume o lugar do bloco original na lista circular (a ordem por endereço continua certa sem percorrer a lista) e entra na árvore com o tamanho novo.

2.5 Fusão (coalescing)

No `free`, o Favo olha o rodapé do bloco anterior e o cabeçalho do seguinte. Há quatro casos, todos em O(1) para a lista e O(log n) para a árvore:

Vizinhos livres	O que acontece
Nenhum	O bloco entra na lista (na posição do seu endereço) e na árvore
Só o anterior	O anterior cresce e absorve o bloco; ele já está na lista, só muda de posição na árvore
Só o seguinte	O bloco absorve o seguinte e ocupa o lugar dele na lista
Os dois	O anterior absorve os dois; o seguinte sai da lista e da árvore

Depois da fusão, nunca existem dois blocos livres vizinhos. O verificador confere isso.

2.6 Blocos grandes

Pedidos de tamanho igual ou maior que `mmap_threshold` (256 KiB no heap global, 32 KiB nas sessões da API) ganham uma arena só para eles, marcada com a flag de bloco grande. No `free`, a arena inteira volta ao sistema com `munmap`.

2.7 Devolução ao sistema

Quando todos os blocos de uma arena `mmap` ficam livres e o heap tem outra arena, ela é devolvida. No `sbrk`, se o último bloco livre passa de duas vezes o tamanho de crescimento, o break é recuado. `favo_heap_trim` faz a mesma coisa sob demanda.

2.8 Detecção de erros

Erro	Como é detectado	Código
Double free	O bloco já está com a assinatura de livre	`FAVO_EDOUBLE`
Ponteiro de fora do heap	O endereço não cai em nenhuma arena	`FAVO_EFOREIGN`
Ponteiro do meio de um bloco ou desalinhado	Assinatura inválida ou endereço não múltiplo de 16	`FAVO_EINVAL`
Escrita além do fim do bloco	O rodapé não bate com o cabeçalho	`FAVO_ECORRUPT`
Falta de memória	O sistema recusou ou o teto `max_bytes` foi atingido	`FAVO_ENOMEM`

No heap global (e no LD_PRELOAD), double free e corrupção abortam o programa com a mensagem do erro, como faz a glibc. Nos heaps criados pelo programa o padrão é devolver o código e seguir.

3. Algoritmos de alocação

Política	Estrutura	Busca	Característica
First-Fit	Lista circular ordenada por endereço	Do início da lista até o primeiro bloco que serve	Simples, tende a concentrar blocos pequenos no começo do heap
Next-Fit	Lista circular	Continua do ponto onde a última busca parou (rover)	Busca curta, espalha os blocos e costuma fragmentar mais
Best-Fit	Árvore rubro-negra por (tamanho, endereço)	Menor bloco com tamanho ≥ pedido, em O(log n)	Menor desperdício, mantém os blocos grandes inteiros

3.1 Ordem da lista

A lista pode ser ordenada por endereço (padrão) ou LIFO. Na ordem por endereço, o bloco liberado é inserido antes do primeiro bloco livre que vem depois dele na memória. Para achar esse vizinho rápido, o Favo anda fisicamente pelos blocos seguintes (até 64) antes de recorrer à lista.

3.2 Árvore rubro-negra

A árvore guarda todos os blocos livres, qualquer que seja a política, para que a troca de política funcione com o heap em uso. As propriedades conferidas pelo verificador são: a raiz é negra, nenhum nó vermelho tem filho vermelho, todo caminho até as folhas tem o mesmo número de nós negros, a ordem (tamanho, endereço) é respeitada e os ponteiros de pai batem.

3.3 Comparação real

Saída de `favo bench --ops 300000` nesta máquina de testes (os números variam conforme o processador):

```
carga aleatoria: 300000 operacoes, blocos de 16 a 4096 bytes, semente 0

alocador               ops/s    tempo (s)     pico (KiB)     fragm.     aprov.
favo first-fit        672846        0.446         4096.0      40.8%      71.0%
favo next-fit        1919132        0.156         5120.0      99.4%      56.8%
favo best-fit        2354926        0.127         4096.0      36.7%      71.0%
malloc sistema      10743466        0.028              -          -          -
```

O malloc da glibc é mais rápido porque usa caches por thread e listas segregadas por tamanho. O Favo prioriza clareza e verificabilidade; a seção 11 mostra como acrescentar listas segregadas.

4. Instalação

4.1 Windows com Docker (recomendado)

1. Instale o **Docker Desktop** (docker.com/products/docker-desktop) e abra o programa até aparecer “Engine running”.
2. Baixe o repositório: botão **Code → Download ZIP** no GitHub e extraia, ou `git clone https://github.com/ravelroot/Custom-Memory-Allocator.git`.
3. Dê dois cliques em `iniciar.cmd`. Na primeira vez a imagem é montada (compila a biblioteca, roda os testes, monta a interface).
4. O navegador abre em **http://127.0.0.1:8080**.
5. Para parar: `docker compose down` na pasta do projeto. Os dados ficam no volume `favo-dados`.

4.2 Linux ou macOS

```
bash iniciar.sh --docker   # com Docker
bash iniciar.sh            # sem Docker: precisa de cmake, cc, python3 e npm
```

Sem Docker, o script compila a biblioteca em `build/`, monta a interface em `web/dist`, cria o ambiente virtual `.venv` e inicia a API.

4.3 Só a biblioteca

```
cmake -S . -B build -DCMAKE_BUILD_TYPE=Release
cmake --build build -j
cd build && ctest --output-on-failure
sudo cmake --install build        # instala favo.h, libfavo, libfavo_preload e o comando
favo
```

Opção do CMake	Padrão	Efeito
FAVO_BUILD_TESTS	ON	Compila os testes
FAVO_BUILD_TOOLS	ON	Compila a linha de comando
FAVO_SANITIZE	OFF	Liga AddressSanitizer e UndefinedBehaviorSanitizer

4.4 Windows nativo

Abra o “Developer PowerShell for VS 2022” na pasta e rode `cmake -S . -B build` e `cmake --build build --config Release`. A DLL fica em `build\Release\favo.dll`. Para a API, defina `FAVO_LIBRARY` com esse caminho. O LD_PRELOAD e o `sbrk` só existem no Linux.

5. Biblioteca em C

5.1 Funções

Função	Descrição
`favo_config_default(cfg)`	Preenche a configuração com os padrões
`favo_heap_create(cfg)`	Cria um heap independente (estrutura em página própria, memória via mmap)
`favo_heap_destroy(h)`	Devolve todas as arenas ao sistema
`favo_heap_reset(h)`	Esvazia o heap e zera os contadores, mantendo a configuração
`favo_heap_alloc(h, n)`	malloc
`favo_heap_calloc(h, q, n)`	calloc com proteção contra overflow de q × n
`favo_heap_realloc(h, p, n)`	realloc: encolhe no lugar, cresce no lugar se o vizinho estiver livre, senão move
`favo_heap_memalign(h, a, n)`	Bloco alinhado a *a* (potência de 2)
`favo_heap_free(h, p)`	free; devolve um código de erro em vez de quebrar
`favo_heap_usable_size(h, p)`	Bytes realmente utilizáveis no bloco
`favo_heap_owns(h, p)`	Diz se o ponteiro pertence ao heap
`favo_heap_set_policy(h, p)`	Troca First/Next/Best-Fit com o heap em uso
`favo_heap_trim(h, folga)`	Devolve memória livre ao sistema
`favo_heap_stats(h, s)`	Estatísticas (seção 5.3)
`favo_heap_check(h, msg, len)`	Verificador completo de integridade
`favo_heap_snapshot / arenas / free_list / tree`	Inspeção dos blocos, arenas, lista e árvore
`favo_malloc, favo_free, favo_calloc, favo_realloc, favo_memalign, favo_posix_memalign, favo_aligned_alloc, favo_usable_size`	Mesmas operações no heap global

Função	Descrição
`favo_bench_run / favo_bench_system`	Benchmark de uma política ou do malloc do sistema

5.2 Configuração (`favo_config`)

Campo	Padrão	Significado
`policy`	`FAVO_FIRST_FIT`	Algoritmo de busca
`source`	`FAVO_SOURCE_AUTO`	AUTO vira mmap em heaps criados e sbrk no global (Linux)
`order`	`FAVO_ORDER_ADDRESS`	Ordem da lista: endereço ou LIFO
`thread_safe`	1	Usa lock em cada operação
`abort_on_error`	0	Aborta em double free, ponteiro inválido e corrupção
`release_empty`	1	Devolve arenas vazias ao sistema
`grow_size`	1 MiB	Tamanho mínimo de cada pedido ao sistema
`mmap_threshold`	256 KiB	A partir deste tamanho o bloco ganha arena própria (0 desliga)
`max_bytes`	0	Teto de memória pedida ao sistema (0 = sem teto)

5.3 Estatísticas (`favo_stats`)

Campo	Significado
`arena_count, arena_bytes, peak_arena_bytes`	Arenas e memória pedida ao sistema (atual e pico)
`used_blocks, used_bytes, requested_bytes`	Blocos ocupados, bytes ocupados e bytes pedidos pelo programa
`free_blocks, free_bytes, largest_free`	Blocos livres e o maior deles
`fragmentation`	1 − maior livre ÷ total livre (0 = nada fragmentado)
`utilization`	Bytes pedidos ÷ memória pedida ao sistema
`splits, coalesces, search_steps`	Divisões, fusões e nós visitados nas buscas
`os_requests, os_releases`	Pedidos e devoluções ao sistema

5.4 Exemplo

```
#include <favo.h>
#include <stdio.h>

int main(void)
{
    favo_config cfg;
    favo_config_default(&cfg);
    cfg.policy = FAVO_BEST_FIT;
    favo_heap *heap = favo_heap_create(&cfg);

    int *v = favo_heap_calloc(heap, 1000, sizeof(int));
    v = favo_heap_realloc(heap, v, 5000 * sizeof(int));

    favo_stats s;
    favo_heap_stats(heap, &s);
    printf("fragmentacao: %.1f%%\n", s.fragmentation * 100);

    if (favo_heap_free(heap, v) != FAVO_OK)
        return 1;
    favo_heap_destroy(heap);
    return 0;
}
```

```
cc exemplo.c -I include -L build -lfavo_static -lpthread -o exemplo
```

6. Substituindo o malloc com LD_PRELOAD

A biblioteca `libfavo_preload.so` exporta `malloc`, `free`, `calloc`, `realloc`, `reallocarray`, `memalign`, `posix_memalign`, `aligned_alloc`, `valloc`, `pvalloc` e `malloc_usable_size`. Carregada antes da libc, ela passa a atender todas as alocações do programa.

```
LD_PRELOAD=./build/libfavo_preload.so python3 -c "print(len([i for i in range(10**6)]))"
FAVO_POLICY=best FAVO_STATS=1 LD_PRELOAD=./build/libfavo_preload.so bash -c "ls / >
/dev/null"
favo[1207] politica=best-fit arenas=1 bytes_arena=1048576 pico=1048576 ocupados=1835
livres=7 ...
```

Variável	Valores	Padrão
`FAVO_POLICY`	first, next, best	first
`FAVO_SOURCE`	sbrk, mmap	sbrk
`FAVO_ORDER`	address, lifo	address
`FAVO_GROW`	bytes, com sufixo K, M ou G	1M
`FAVO_MMAP_THRESHOLD`	bytes (0 desliga)	256K
`FAVO_MAX_BYTES`	bytes (0 = sem teto)	0
`FAVO_ABORT`	0 ou 1	1
`FAVO_STATS`	1 imprime estatísticas ao sair	0

Ponteiros que não pertencem ao heap (por exemplo, memória do carregador dinâmico) são ignorados no `free`. A versão com preload registra `pthread_atfork` para que o processo filho não herde o lock preso.

7. Linha de comando

Comando	O que faz
`favo demo [--policy P]`	Mostra passo a passo alocação, buracos, escolha do bloco e fusão
`favo run arquivo [--policy P] [--order O] [--check] [--map] [--blocks]`	Executa um trace; `--check` verifica o heap depois de cada comando
`favo compare arquivo`	Executa o trace nos três algoritmos e compara
`favo bench [--ops N] [--seed S] [--min B] [--max B]`	Carga aleatória contra o malloc do sistema
`favo gen [--ops N] [--seed S] [--max B] [--out arquivo]`	Gera um trace aleatório

7.1 Formato do trace

Linha	Efeito
`a id bytes`	malloc
`c id qtd bytes`	calloc
`m id alinhamento bytes`	memalign
`r id bytes`	realloc (0 libera)
`f id`	free
linha vazia, `#` ou `;`	ignorada

7.2 Exemplo de saída

```
trace            traces/fragmentacao.trace (160 comandos)
executados       160 (0 falhas, 0 ignorados) em 0.000 s
politica         best-fit
arenas           1 (64.0 KiB, pico 64.0 KiB)
blocos ocupados  64 (24.1 KiB, pedidos 22.1 KiB)
blocos livres    15 (39.8 KiB, maior 32.1 KiB)
fragmentacao     19.4%
aproveitamento   34.5%
operacoes        112 alocacoes, 48 liberacoes, 0 realocacoes, 0 falhas
estrutura        110 divisoes, 32 fusoes, 348 passos de busca
sistema          1 pedidos, 0 devolucoes

arena 0  mmap    64.0 KiB
|#######.........###################.................................|
legenda: # ocupado  . livre  = bloco grande (mmap proprio)
verificacao      ok
```

```
1) oito blocos alocados em sequencia
arena 0  mmap    16.0 KiB  |[#####[#######[###
[#############.............................|
legenda: # ocupado  . livre  = bloco grande (mmap proprio)

2) blocos 1, 3 e 5 liberados: buracos no meio do heap
arena 0  mmap    16.0 KiB  |[###..[#######[###
[#############.............................|
legenda: # ocupado  . livre  = bloco grande (mmap proprio)

3) pedido de 100 bytes com best-fit: escolhido o endereco 0x7f56ca966e60
arena 0  mmap    16.0 KiB  |[###..[#######[###
[#############.............................|
legenda: # ocupado  . livre  = bloco grande (mmap proprio)
```

8. Painel web

8.1 Heap

Cada sessão é um heap isolado. À esquerda ficam a sessão, o algoritmo (com troca a qualquer momento), os formulários de malloc, calloc e memalign, os tamanhos rápidos e a execução de traces. No centro, as estatísticas, o mapa do heap (cada arena é uma barra e cada bloco ocupa uma largura proporcional ao tamanho), o bloco selecionado, a lista circular, a árvore rubro-negra, os blocos alocados e o histórico gravado no SQLite.



Tela Heap: mapa, bloco selecionado com gravação e leitura de bytes, estruturas de busca e histórico.

Clique num bloco ocupado para ver endereço, alinhamento, tamanho pedido, tamanho útil e tamanho total; gravar texto dentro dele; ler os bytes em hexadecimal; realocar ou liberar. O selo “heap consistente” mostra o resultado do verificador depois de cada operação. A atualização chega pelo WebSocket, então duas abas abertas na mesma sessão ficam sincronizadas.



O mesmo painel no tema escuro (botão Tema no topo).

8.2 Comparar

Cole um trace ou gere um aleatório e rode. O mesmo trace é executado em três heaps novos, um por algoritmo, e o painel mostra o mapa final e os números de cada um lado a lado.



Comparação do mesmo trace em First-Fit, Next-Fit e Best-Fit.

8.3 Desempenho

Roda o benchmark dentro da biblioteca em C (sem passar pelo Python a cada operação) e guarda o resultado no banco.



Operações por segundo, pico de memória, fragmentação e aproveitamento de cada alocador.

8.4 Traces

Editor para salvar traces com nome, abrir, excluir e mandar para a comparação.



Traces salvos no SQLite.

9. API REST e WebSocket

A API está em `api/favo_api`. Ela carrega a `libfavo` com ctypes, mantém as sessões em memória e grava o histórico no SQLite. A documentação interativa fica em `/docs`.

Método e rota	Corpo	Resposta
`POST /api/sessions`	name, policy, order, grow_size, mmap_threshold, max_bytes	Sessão criada
`GET /api/sessions`	-	Sessões abertas
`DELETE /api/sessions/{id}`	-	204
`GET /api/sessions/{id}/heap`	-	stats, check, arenas, blocks, free_list, tree, allocations
`POST /api/sessions/{id}/alloc`	size, label	handle, address, usable
`POST /api/sessions/{id}/calloc`	count, size, label	idem
`POST /api/sessions/{id}/memalign`	alignment, size, label	idem
`POST /api/sessions/{id}/realloc`	handle, size	idem, com o endereço novo
`POST /api/sessions/{id}/free`	handle	freed
`POST /api/sessions/{id}/free-all`	-	ok
`POST /api/sessions/{id}/write`	handle, text	written, usable
`GET /api/sessions/{id}/read/{handle}`	length	hex, text
`PUT /api/sessions/{id}/policy`	policy	Sessão
`POST /api/sessions/{id}/trim`, `/reset`, `/trace`	content (trace)	Resultado
`GET /api/sessions/{id}/operations`	limit	Histórico
`POST /api/compare`	content	Resultado por política
`POST /api/benchmarks`, `GET /api/benchmarks`	operations, seed, min_size, max_size	Execução com resultados
`GET/POST /api/traces`, `GET/DELETE /api/traces/{id}`, `POST /api/traces/generate`	name, content	Traces

Erros do alocador voltam com status 409 e o campo `code` (1 memória insuficiente, 2 inválido, 3 double free, 4 ponteiro de fora, 5 corrompido). Validação de entrada volta com 422.

```
curl -X POST http://127.0.0.1:8080/api/sessions -H "Content-Type: application/json" -d
'{"name":"teste","policy":"best-fit"}'
curl -X POST http://127.0.0.1:8080/api/sessions/ID/alloc -H "Content-Type: application/json"
-d '{"size":256,"label":"buffer"}'
curl http://127.0.0.1:8080/api/sessions/ID/heap
```

9.1 WebSocket

Conecte em `/ws/sessions/{id}`. A primeira mensagem é `{"type":"snapshot","data":...}` com o mesmo conteúdo de `/heap`, e uma nova chega a cada operação na sessão. Envie `ping` para receber `pong` ou `refresh` para pedir o estado de novo.

10. Configuração

Variável	Padrão	Uso
`FAVO_LIBRARY`	procura em build/	Caminho da libfavo.so ou favo.dll
`FAVO_DATABASE`	data/favo.db	Arquivo do SQLite
`FAVO_WEB_DIR`	web/dist	Interface compilada
`FAVO_HOST`	127.0.0.1	Endereço da API (0.0.0.0 no Docker)
`FAVO_PORT`	8080	Porta
`FAVO_MAX_SESSIONS`	32	Sessões abertas ao mesmo tempo; sessões paradas por 2 h são fechadas
`FAVO_CORS`	vazio	Origens liberadas para outro front-end

Cada sessão tem teto de memória (`max_bytes`, padrão 16 MiB, máximo 256 MiB) e cada bloco pedido pela API é limitado a 8 MiB, para que o servidor não seja esgotado por engano.

11. Como estender o Favo

11.1 Nova política: Worst-Fit

1. Em `include/favo.h`, acrescente `FAVO_WORST_FIT = 3` em `favo_policy`.
2. Em `src/heap.c`, ajuste `normalize` para aceitar 3 e, em `find_fit`, desça sempre para a direita na árvore:

```
if (h->cfg.policy == FAVO_WORST_FIT) {
    favo_node *n = h->root;
    while (n && n->right)
        n = n->right;
    return n && blk_size(&n->head) >= need ? n : NULL;
}
```

3. Inclua o nome em `favo_policy_name` (`src/global.c`), em `POLICIES` (`api/favo_api/binding.py`), no tipo `Policy` da API e em `POLICIES` do `web/src/main.ts`.
4. Acrescente um teste em `tests/test_policies.c` e rode `make test`.

11.2 Listas segregadas por tamanho

Troque `favo_node *list` da `struct favo_heap` por `favo_node *lists[N]`, escolha o índice por `log2(tamanho)` em `favo_list_insert`, `favo_list_remove` e `favo_list_replace`, e faça `find_fit` começar na lista da classe do pedido e subir para as maiores. Ajuste o verificador em `src/inspect.c` para percorrer todas as listas.

11.3 Nova origem de memória

Escreva a função em `src/os.c` (por exemplo `mmap` com `MAP_HUGETLB`), acrescente a constante em `favo_source` e trate em `grow` no `src/heap.c`.

11.4 Nova rota da API

Dentro de `create_app` em `api/favo_api/app.py`, crie a função com o decorador do FastAPI. Se a rota alterar o heap, use `await mutate(sessao, "tipo", acao)`: ela segura o lock da sessão, grava a operação no SQLite (acrescente o tipo novo no `CHECK` da tabela `operations` com uma migração) e avisa o WebSocket. Escreva o teste em `api/tests/test_api.py`.

11.5 Nova tela

Em `web/src/main.ts`, acrescente o valor em `View`, o botão em `shell()` e a função que devolve o HTML da tela. Use `heapMap`, `treeSvg` e os formatadores de `web/src/render.ts`. Rode `npm run dev` (porta 5173, com proxy para a API) para desenvolver e `npm run build` para gerar a versão final.

12. Testes

Suíte	Comando	Cobre
test_basic	`ctest -R basic`	malloc/free, alinhamento de 0 a 300 bytes, calloc, realloc no lugar e movendo, fusão, memalign até 64 KiB, blocos grandes, erros, teto de memória, trim, heap global
test_policies	`ctest -R policies`	Bloco escolhido por cada política, troca de política, LIFO, inspeção, benchmark
test_stress	`ctest -R stress`	150 mil operações aleatórias por política com conferência do conteúdo e do heap, sbrk
test_threads	`ctest -R threads`	Oito threads no mesmo heap em cada política
preload	`ctest -R preload`	ls, sort, awk, bash e python3 rodando com LD_PRELOAD
traces	`ctest -R traces`	Todos os traces de exemplo nas três políticas com verificação a cada comando
API	`make api`	Sessões, memória, leitura e escrita, políticas, teto, traces, comparação, benchmark e WebSocket

```
make test       # C, preload e traces
make asan       # AddressSanitizer + UBSan
make valgrind   # Valgrind
make api        # pytest
```

13. Solução de problemas

Sintoma	Causa provável	Solução
`biblioteca do Favo não encontrada` ao iniciar a API	A libfavo não foi compilada ou está em outro lugar	Rode `cmake --build build` ou defina `FAVO_LIBRARY`
`iniciar.cmd` avisa que o Docker não foi encontrado	Docker Desktop não instalado ou fechado	Instale e abra o Docker Desktop, espere “Engine running”
Porta 8080 em uso	Outro programa usa a porta	Troque `127.0.0.1:8080` no `docker-compose.yml` ou use `FAVO_PORT`
Programa aborta com `favo: double free detectado`	O programa liberou o mesmo ponteiro duas vezes	Corrija o programa; para investigar, rode com `FAVO_ABORT=0`
`heap corrompido`	Escrita além do tamanho pedido	Rode o programa com `make asan` ou Valgrind para achar a escrita
Status 409 “memória insuficiente” na API	Teto `max_bytes` da sessão atingido	Crie a sessão com teto maior ou libere blocos
Painel sem atualização ao vivo	WebSocket bloqueado por proxy	Acesse direto em 127.0.0.1:8080; a página continua funcionando por requisições
LD_PRELOAD sem efeito	Binário estático ou com setuid	Só funciona com binários dinâmicos sem setuid